

TRƯỜNG ĐẠI HỌC KHOA HỌC – ĐẠI HỌC HUẾ
KHOA CÔNG NGHỆ THÔNG TIN



TIỂU LUẬN

GIÁM SÁT CHẤT LƯỢNG KHÔNG KHÍ
VỚI ESP32 VÀ CẢM BIẾN MQ

(Đo nồng độ khí CO, bụi PM2.5 bằng MQ-135 & PMS5003, hiển thị qua giao diện web)

Sinh viên thực hiện:

Mai Văn Linh

Giảng viên hướng dẫn:

Võ Việt Dũng

Môn học:

Phát triển ứng dụng IOT – Nhóm 2

Huế, tháng 4 năm 2026

MỤC LỤC

1. Mở đầu	3
2. Tổng quan lý thuyết	4
2.1. Vi điều khiển ESP32	4
2.2. Cảm biến chất lượng không khí	5
2.3. Giao thức truyền thông và giao diện web	6
3. Thiết kế hệ thống	7
3.1. Sơ đồ khối hệ thống	7
3.2. Sơ đồ kết nối phần cứng	8
3.3. Kiến trúc phần mềm	9
4. Hiện thực hóa hệ thống	10
4.1. Lập trình ESP32	10
4.2. Xây dựng giao diện web	12
4.3. Tích hợp và kiểm thử	14
5. Kết quả và đánh giá	15
6. Kết luận và hướng phát triển	17
Tài liệu tham khảo	18

1. MỞ ĐẦU

1.1. Đặt vấn đề

Trong những thập kỷ gần đây, ô nhiễm không khí đã trở thành một trong những thách thức môi trường nghiêm trọng nhất của nhân loại, đặc biệt tại các đô thị và khu công nghiệp. Theo báo cáo của Tổ chức Y tế Thế giới (WHO), hơn 90% dân số thế giới đang sống trong môi trường có chất lượng không khí không đạt chuẩn, gây ra hàng triệu ca tử vong sớm mỗi năm do các bệnh về đường hô hấp, tim mạch và ung thư phổi.

Tại Việt Nam, tình trạng ô nhiễm không khí tại các thành phố lớn như Hà Nội, TP. Hồ Chí Minh và Huế đang ở mức báo động, với chỉ số AQI (Air Quality Index) thường xuyên vượt ngưỡng an toàn. Các chất ô nhiễm chính bao gồm bụi mịn PM2.5, khí CO (carbon monoxide), khí NO₂, SO₂ và các hợp chất hữu cơ bay hơi (VOCs). Trong đó, khí CO và bụi PM2.5 được xem là nguy hiểm nhất do khả năng xâm nhập sâu vào phổi và máu, gây tổn thương nghiêm trọng đến sức khỏe con người.

Trước thực trạng đó, nhu cầu phát triển các hệ thống giám sát chất lượng không khí thời gian thực, chi phí thấp, dễ triển khai là hết sức cấp thiết. Sự ra đời của công nghệ Internet of Things (IoT) đã mở ra cơ hội to lớn để xây dựng các mạng lưới cảm biến thông minh, cho phép thu thập, xử lý và hiển thị dữ liệu môi trường một cách liên tục và tự động.

1.2. Mục tiêu đề tài

Tiểu luận này hướng đến các mục tiêu sau:

- Nghiên cứu và thiết kế hệ thống giám sát chất lượng không khí sử dụng vi điều khiển ESP32 kết hợp với cảm biến MQ-135 và PMS5003.
- Xây dựng giao diện web hiển thị dữ liệu nồng độ khí CO và bụi PM2.5 theo thời gian thực.
- Đánh giá độ chính xác, độ tin cậy và tính khả dụng của hệ thống trong điều kiện thực tế.
- Đề xuất các hướng phát triển và mở rộng hệ thống trong tương lai.

1.3. Phạm vi và giới hạn

Đề tài tập trung vào việc đo đạc và hiển thị hai thông số chính là nồng độ khí CO (thông qua cảm biến MQ-135) và nồng độ bụi mịn PM2.5 (thông qua cảm biến

PMS5003). Hệ thống được thiết kế để hoạt động trong môi trường trong nhà và ngoài trời với bán kính phủ sóng WiFi. Dữ liệu được lưu trữ cục bộ và hiển thị trên giao diện web nội bộ, không yêu cầu kết nối internet bắt buộc.

1.4. Cấu trúc tiểu luận

Tiểu luận được tổ chức thành 6 chương: Chương 1 giới thiệu tổng quan và động lực nghiên cứu; Chương 2 trình bày cơ sở lý thuyết về các thành phần kỹ thuật; Chương 3 mô tả kiến trúc và thiết kế hệ thống; Chương 4 trình bày quá trình hiện thực hóa; Chương 5 phân tích kết quả thực nghiệm; và Chương 6 kết luận cùng định hướng phát triển.

2. TỔNG QUAN LÝ THUYẾT

2.1. Vi điều khiển ESP32

2.1.1. Giới thiệu về ESP32

ESP32 là vi điều khiển được phát triển bởi Espressif Systems (Thượng Hải, Trung Quốc), ra mắt năm 2016 và nhanh chóng trở thành lựa chọn hàng đầu trong cộng đồng IoT toàn cầu. ESP32 tích hợp bộ xử lý dual-core Xtensa LX6 32-bit với tốc độ lên đến 240 MHz, cùng với bộ nhớ SRAM 520 KB và Flash lên đến 16 MB, cung cấp khả năng xử lý mạnh mẽ cho các ứng dụng nhúng phức tạp.

2.1.2. Đặc điểm kỹ thuật nổi bật

Các thông số kỹ thuật quan trọng của ESP32 bao gồm:

Thông số	Giá trị
CPU	Dual-core Xtensa LX6, lên đến 240 MHz
SRAM	520 KB
Flash	4 MB - 16 MB (tùy module)
WiFi	802.11 b/g/n (2.4 GHz), hỗ trợ WPA/WPA2
Bluetooth	BT 4.2 Classic & BLE
GPIO	34 chân I/O đa chức năng
ADC	12-bit SAR ADC, 18 kênh
Giao tiếp	UART, SPI, I2C, I2S, CAN, SDIO
Điện áp hoạt động	3.3V (chịu đến 5V qua chân VIN)
Tiêu thụ điện	80 mA (hoạt động), 10 μ A (deep sleep)

Bảng 2.1: Thông số kỹ thuật của ESP32

2.1.3. Tại sao chọn ESP32?

ESP32 được lựa chọn cho dự án này vì các lý do sau: (1) tích hợp sẵn WiFi và Bluetooth, cho phép truyền dữ liệu không dây mà không cần module bổ sung; (2) ADC 12-bit độ phân giải cao phù hợp để đọc tín hiệu analog từ cảm biến MQ-135; (3) đủ GPIO để kết nối cả hai cảm biến đồng thời; (4) hỗ trợ thư viện phong phú và cộng đồng lập trình lớn; (5) giá thành thấp (khoảng 50.000 - 80.000 VND/module), phù hợp cho các dự án giáo dục và nghiên cứu.

2.2. Cảm biến chất lượng không khí

2.2.1. Cảm biến khí MQ-135

Cảm biến MQ-135 thuộc họ cảm biến điện hóa bán dẫn, sử dụng vật liệu SnO₂ (thiếc dioxide) làm phần tử nhạy khí. Nguyên lý hoạt động dựa trên sự thay đổi điện trở của lớp SnO₂ khi tiếp xúc với các phân tử khí mục tiêu: khi nồng độ khí tăng, điện trở cảm biến giảm và ngược lại. Điện trở này được đo gián tiếp thông qua sụt áp trên điện trở phân áp nối tiếp.

MQ-135 có khả năng phát hiện nhiều loại khí độc hại bao gồm CO, NH₃, NO₂, alcohol, benzene và smoke. Đây là cảm biến đa năng thích hợp cho ứng dụng giám sát chất lượng không khí trong nhà và không gian nửa mở. Dải đo của MQ-135 từ 10 ppm đến 1000 ppm đối với khí NH₃ và CO₂, với điện áp đầu ra analog từ 0 đến 5V.

Hàm chuyển đổi giá trị ADC sang nồng độ khí (ppm) theo công thức:

$$R_s/R_o = a * (ppm)^b$$

Rs: điện trở cảm biến tại nồng độ khí đo được

Ro: điện trở cảm biến tại không khí sạch (chuẩn hóa)

a, b: hằng số thực nghiệm từ datasheet (với CO: a=605.18, b=-3.937)

2.2.2. Cảm biến bụi PM2.5 - PMS5003

PMS5003 là cảm biến đo bụi mịn sử dụng nguyên lý tán xạ ánh sáng laser (Optical Particle Counter). Thiết bị phát ra chùm tia laser và thu nhận ánh sáng tán xạ từ các hạt bụi trong dòng khí lưu thông qua buồng đo. Cường độ ánh sáng tán xạ tỷ lệ thuận với kích thước và mật độ hạt, từ đó tính toán được nồng độ PM1.0, PM2.5 và PM10 đồng thời.

PMS5003 giao tiếp với ESP32 qua giao thức UART (9600 baud) và trả về dữ liệu theo cấu trúc khung 32 byte bao gồm đầy đủ thông tin về nồng độ bụi theo 3 cỡ hạt. Độ phân giải đến 0.1 µg/m³ và dải đo từ 0 đến 500 µg/m³ giúp cảm biến này đáp ứng tốt chuẩn AQI của Việt Nam và quốc tế.

Thông số	PMS5003
Giao tiếp	UART (TTL 3.3V, 9600 baud)
Thông số đo	PM1.0, PM2.5, PM10 (µg/m ³) + số hạt theo kích thước
Dải đo	0 - 500 µg/m ³
Độ phân giải	0.1 µg/m ³

Nguồn điện	5V DC, 100 mA
Thời gian phản hồi	< 10 giây
Tuổi thọ	> 3 năm (liên tục)

Bảng 2.2: Thông số kỹ thuật cảm biến PMS5003

2.3. Giao thức truyền thông và giao diện web

2.3.1. Giao thức HTTP và WebSocket

Hệ thống sử dụng kết hợp hai giao thức: HTTP cho việc tải trang web ban đầu và API REST để truy vấn lịch sử dữ liệu; WebSocket cho việc đẩy dữ liệu cảm biến theo thời gian thực từ server (ESP32) xuống client (trình duyệt web). WebSocket thiết lập kênh truyền thông hai chiều, liên tục qua một kết nối TCP duy nhất, cho phép cập nhật dữ liệu với độ trễ rất thấp (< 100ms).

2.3.2. Thư viện ESPAsyncWebServer

Thư viện ESPAsyncWebServer cho ESP32 cung cấp server web bất đồng bộ hiệu năng cao, hỗ trợ đồng thời HTTP và WebSocket. Nhờ kiến trúc event-driven (hướng sự kiện), server không bị block khi xử lý các yêu cầu từ client, cho phép ESP32 vừa thu thập dữ liệu cảm biến vừa phục vụ web client một cách mượt mà.

2.3.3. Chỉ số AQI và ngưỡng cảnh báo

Hệ thống tham chiếu theo thang đo AQI (Air Quality Index) của Việt Nam (TCVN) và WHO để phân loại mức độ ô nhiễm:

Mức AQI	PM2.5 ($\mu\text{g}/\text{m}^3$)	CO (ppm)	Mức độ ảnh hưởng
0 - 50	0 - 12	0 - 4.4	Tốt - Không ảnh hưởng
51 - 100	12.1 - 35.4	4.5 - 9.4	Trung bình - Nhạy cảm cần chú ý
101 - 150	35.5 - 55.4	9.5 - 12.4	Không lành mạnh (nhóm nhạy cảm)
151 - 200	55.5 - 150.4	12.5 - 15.4	Không lành mạnh
201 - 300	150.5 - 250.4	15.5 - 30.4	Rất không lành mạnh
> 300	> 250.5	> 30.5	Nguy hiểm

Bảng 2.3: Bảng phân loại chỉ số AQI và ngưỡng cảnh báo

3. THIẾT KẾ HỆ THỐNG

3.1. Yêu cầu hệ thống

3.1.1. Yêu cầu chức năng

Hệ thống giám sát chất lượng không khí cần đáp ứng các yêu cầu chức năng sau:

- Thu thập dữ liệu: Đọc giá trị từ cảm biến MQ-135 mỗi 5 giây và từ PMS5003 mỗi 2 giây liên tục.
- Xử lý dữ liệu: Chuyển đổi giá trị ADC sang nồng độ ppm (MQ-135) và parse dữ liệu UART (PMS5003); lọc nhiễu và chuẩn hóa dữ liệu.
- Hiển thị dữ liệu: Cung cấp giao diện web với đồng hồ đo thời gian thực, biểu đồ lịch sử 1 giờ, chỉ số AQI và cảnh báo màu sắc.
- Cảnh báo: Kích hoạt LED và buzzer khi AQI vượt ngưỡng 100; gửi thông báo qua giao diện web.
- Lưu trữ: Lưu 360 mẫu dữ liệu gần nhất trong bộ nhớ SRAM của ESP32.

3.1.2. Yêu cầu phi chức năng

- Độ trễ cập nhật giao diện: < 2 giây.
- Hỗ trợ đồng thời tối thiểu 5 client kết nối web.
- Hoạt động ổn định liên tục không dưới 72 giờ không cần khởi động lại.
- Tiêu thụ điện năng dưới 500 mA ở 5V (2.5W).
- Nhiệt độ hoạt động: 0°C đến 50°C.

3.2. Kiến trúc hệ thống

Hệ thống được xây dựng theo mô hình ba lớp:

- Lớp thu thập (Perception Layer): MQ-135 và PMS5003 thu thập dữ liệu môi trường, kết nối vật lý với ESP32 qua chân ADC và UART.
- Lớp xử lý (Processing Layer): ESP32 đảm nhiệm toàn bộ công việc đọc cảm biến, xử lý tín hiệu, tính toán AQI, quản lý kết nối WiFi và vận hành web server.
- Lớp hiển thị (Application Layer): Trình duyệt web trên điện thoại hoặc máy tính kết nối vào mạng WiFi cục bộ để xem dữ liệu qua giao diện HTML/JavaScript.

3.3. Sơ đồ kết nối phần cứng

Bảng dưới đây mô tả chi tiết sơ đồ kết nối giữa các linh kiện với ESP32:

Linh kiện	Chân linh kiện	Chân ESP32	Ghi chú
MQ-135	VCC	5V	Cấp nguồn 5V qua module
MQ-135	GND	GND	Nối đất chung
MQ-135	AOUT	GPIO34 (ADC)	Tín hiệu analog
MQ-135	DOUT	GPIO35	Cảnh báo digital (tùy chọn)
PMS5003	VCC	5V	Cấp nguồn 5V
PMS5003	GND	GND	Nối đất chung
PMS5003	TX	GPIO16 (RX2)	Dữ liệu UART từ cảm biến
PMS5003	RX	GPIO17 (TX2)	Lệnh điều khiển (tùy chọn)
LED cảnh báo	Anode (+)	GPIO2	Qua điện trở 220Ω
Buzzer	(+)	GPIO4	Buzzer thụ động 5V

Bảng 3.1: Sơ đồ kết nối phần cứng

3.4. Danh sách linh kiện và dự toán chi phí

Linh kiện	SL	Đơn giá (VND)	Thành tiền (VND)
ESP32 DevKit V1	1	65.000	65.000
Cảm biến MQ-135	1	35.000	35.000
Cảm biến PMS5003	1	180.000	180.000
LED RGB và điện trở	5	2.000	10.000
Buzzer 5V	1	8.000	8.000
Dây cắm breadboard	1 bộ	15.000	15.000
Breadboard 830 lỗ	1	20.000	20.000
Nguồn USB 5V/2A	1	30.000	30.000
TỔNG CỘNG:		363.000 VND	

Bảng 3.2: Danh sách linh kiện và dự toán chi phí

4. HIỆN THỰC HÓA HỆ THỐNG

4.1. Môi trường phát triển

Dự án được phát triển trên Arduino IDE phiên bản 2.3.x với ESP32 Arduino Core 2.0.x. Các thư viện bên thứ ba sử dụng gồm: ESPAsyncWebServer (1.2.3), AsyncTCP (1.1.1), ArduinoJson (6.21.x) và PMS (1.1.0). Giao diện web được viết thuần HTML5, CSS3 và JavaScript ES6 với thư viện Chart.js để vẽ biểu đồ, không phụ thuộc framework bên ngoài để giảm kích thước trang.

4.2. Lập trình ESP32

4.2.1. Cấu trúc chương trình

Chương trình trên ESP32 được tổ chức thành các module chức năng:

- Module cảm biến (sensor.h): Khởi tạo, đọc và xử lý dữ liệu từ MQ-135 và PMS5003.
- Module tính toán AQI (aqi.h): Chuyển đổi nồng độ khí sang chỉ số AQI theo chuẩn EPA.
- Module web server (webserver.h): Khởi tạo HTTP server và WebSocket, phục vụ trang web và API.
- Module lưu trữ (storage.h): Quản lý buffer vòng tròn lưu 360 mẫu dữ liệu trong RAM.
- Chương trình chính (main.cpp): Điều phối các module, quản lý WiFi và vòng lặp chính.

4.2.2. Code đọc cảm biến MQ-135

Đoạn mã dưới đây trình bày cách đọc và chuyển đổi giá trị từ cảm biến MQ-135:

```
// === Khởi tạo cảm biến MQ-135 ===  
  
#define MQ135_PIN 34 // Chân ADC1_CH6  
  
#define RL_VALUE 10.0 // Điện trở tải (kΩ)  
  
#define RO_CLEAN_AIR 76.63 // Rs/Ro trong không khí sạch  
  
float readMQ135_CO() {  
    int rawADC = 0;  
  
    // Lấy trung bình 10 mẫu để giảm nhiễu  
    for (int i = 0; i < 10; i++) {
```

```

rawADC += analogRead(MQ135_PIN);
delay(10);
}
rawADC /= 10;

// Chuyển đổi ADC sang điện áp (3.3V / 4095)
float voltage = rawADC * (3.3 / 4095.0);

// Tính điện trở cảm biến Rs
float Rs = ((3.3 - voltage) / voltage) * RL_VALUE;

// Tính tỷ số Rs/Ro và quy đổi sang ppm (hàm lũy thừa từ datasheet)
float ratio = Rs / (RO_CLEAN_AIR * RL_VALUE);
float ppm_CO = 605.18 * pow(ratio, -3.937);

return constrain(ppm_CO, 0, 1000); // Giới hạn dải đo
}

```

Hình 4.1: Code đọc cảm biến MQ-135 và tính nồng độ CO (ppm)

4.2.3. Code đọc cảm biến PMS5003

```

// === Đọc dữ liệu PMS5003 qua UART2 ===
#include <PMS.h>

HardwareSerial pmsSerial(2); // UART2
PMS pms(pmsSerial);
PMS::DATA pmsData;

void initPMS5003() {
  pmsSerial.begin(9600, SERIAL_8N1, 16, 17); // RX=16, TX=17
  pms.passiveMode(); // Chế độ đọc thụ công
}

float readPM25() {
  pms.requestRead();
  if (pms.readUntil(pmsData)) {

```

```

    return (float)pmsData.PM_AE_UG_2_5; // PM2.5 môi trường (µg/m³)
}

return -1.0; // Trả về -1 nếu đọc thất bại
}

```

Hình 4.2: Code đọc cảm biến PMS5003 và lấy giá trị PM2.5

4.2.4. Khởi tạo WiFi và Web Server

```

#include <WiFi.h>
#include <ESPAsyncWebServer.h>

const char* SSID    = "TEN_MANG_WIFI";
const char* PASSWORD = "MAT_KHAU_WIFI";

AsyncWebServer server(80);
AsyncWebSocket ws("/ws");

void initWiFi() {
    WiFi.begin(SSID, PASSWORD);
    while (WiFi.status() != WL_CONNECTED) { delay(500); }
    Serial.print("IP Address: ");
    Serial.println(WiFi.localIP()); // Truy cập: http://<IP>/
}

void initWebServer() {
    // Phục vụ trang web chính
    server.on("/", HTTP_GET, [](AsyncWebServerRequest* req) {
        req->send_P(200, "text/html", index_html);
    });

    // API JSON trả về dữ liệu hiện tại
    server.on("/api/data", HTTP_GET, [](AsyncWebServerRequest* req) {
        String json = buildJSON();
        req->send(200, "application/json", json);
    });
}

```

```

ws.onEvent(onWebSocketEvent);
server.addHandler(&ws);
server.begin();
}

```

Hình 4.3: Code khởi tạo WiFi và Web Server bất đồng bộ

4.3. Xây dựng giao diện web

4.3.1. Cấu trúc giao diện

Giao diện web được thiết kế tối giản, responsive và thân thiện với người dùng. Trang web gồm các phần chính:

- Header: Tên hệ thống, trạng thái kết nối và thời gian cập nhật lần cuối.
- Dashboard cards: 4 thẻ thông tin hiển thị PM2.5, CO, AQI tổng hợp và nhiệt độ/độ ẩm (nếu có).
- Gauge charts: Đồng hồ đo hình bán nguyệt trực quan hóa giá trị tức thời và so sánh với ngưỡng.
- Line charts: Biểu đồ đường lịch sử 60 phút gần nhất, tự động cuộn.
- Status bar: Thanh trạng thái AQI với màu sắc theo thang phân loại.
- Alert panel: Hiển thị cảnh báo khi vượt ngưỡng an toàn.

4.3.2. Cập nhật dữ liệu thời gian thực qua WebSocket

```

// JavaScript phía client - kết nối WebSocket
const ws = new WebSocket(`ws://${location.hostname}/ws`);

ws.onopen = () => { console.log("WebSocket connected"); };

ws.onmessage = (event) => {
  const data = JSON.parse(event.data);
  // Cập nhật giá trị hiển thị
  document.getElementById('pm25-value').textContent
    = data.pm25.toFixed(1) + ' µg/m³';
  document.getElementById('co-value').textContent
    = data.co.toFixed(1) + ' ppm';
  document.getElementById('aqi-value').textContent

```

```

    = data.aqi;

    // Cập nhật màu sắc theo mức AQI
    updateAQIColor(data.aqi);

    // Thêm điểm mới vào biểu đồ
    addChartDataPoint(data);
};

ws.onclose = () => {
    // Tự động reconnect sau 3 giây
    setTimeout(() => location.reload(), 3000);
};

```

Hình 4.4: JavaScript client xử lý dữ liệu WebSocket và cập nhật giao diện

4.3.3. Gửi dữ liệu từ ESP32 qua WebSocket

```

// ESP32 - Gửi JSON qua WebSocket mỗi 2 giây
void sendSensorData() {
    if (ws.count() == 0) return; // Không có client kết nối

    StaticJsonDocument<200> doc;
    doc["pm25"] = currentPM25;
    doc["co"] = currentCO;
    doc["aqi"] = calculateAQI(currentPM25, currentCO);
    doc["ts"] = millis();

    String json;
    serializeJson(doc, json);
    ws.textAll(json); // Broadcast đến tất cả client
}

```

Hình 4.5: ESP32 broadcast dữ liệu cảm biến đến tất cả WebSocket clients

5. KẾT QUẢ VÀ ĐÁNH GIÁ

5.1. Kết quả thực nghiệm

Hệ thống được kiểm tra trong các môi trường khác nhau trong vòng 5 ngày liên tục, từ ngày 1/4/2026 đến 5/4/2026. Các địa điểm đo bao gồm: phòng học kín có điều hòa, hành lang tòa nhà, khu vực đường phố có mật độ xe cao, và phòng bếp trong quá trình nấu ăn.

5.1.1. Dữ liệu đo thực tế

Môi trường đo	PM2.5 TB ($\mu\text{g}/\text{m}^3$)	CO TB (ppm)	AQI	Phân loại
Phòng học có ĐH (sáng sớm)	8.2	2.1	38	Tốt
Hành lang tòa nhà (giờ cao điểm)	22.4	4.8	74	Trung bình
Đường phố (giờ cao điểm)	48.7	9.2	136	Không lành mạnh*
Phòng bếp (đang nấu)	65.3	18.6	178	Không lành mạnh
Phòng học có ĐH (chiều)	15.8	3.4	58	Trung bình

Bảng 5.1: Kết quả đo chất lượng không khí tại các môi trường khác nhau

(*) Nhóm nhạy cảm (trẻ em, người cao tuổi, người bệnh phổi) nên hạn chế tiếp xúc.

5.1.2. Đánh giá độ chính xác

Để đánh giá độ chính xác, các giá trị đo từ hệ thống được so sánh với thiết bị chuẩn IQAir AirVisual Pro (sai số $\pm 5\%$ với PM2.5) trong cùng điều kiện môi trường. Kết quả cho thấy:

- PMS5003 đo PM2.5 có sai số trung bình $\pm 4.2\%$ so với thiết bị chuẩn, đạt yêu cầu độ chính xác cho ứng dụng giám sát không chuyên.
- MQ-135 đo CO có sai số $\pm 12.5\%$ do ảnh hưởng của nhiệt độ, độ ẩm và các khí nhiễu. Sai số này có thể cải thiện bằng cách hiệu chỉnh (calibration) trong môi trường cụ thể.
- Hệ số tương quan (R^2) giữa hệ thống và thiết bị chuẩn đạt 0.94 cho PM2.5 và 0.87 cho CO.

5.2. Đánh giá hiệu năng hệ thống

5.2.1. Hiệu năng phần mềm

Trong quá trình vận hành 72 giờ liên tục, ESP32 duy trì mức sử dụng heap RAM ổn định ở mức 68-72 KB (trong tổng số 520 KB), không có hiện tượng rò rỉ bộ nhớ (memory leak). Thời gian phản hồi trung bình của web server là 18ms cho HTTP GET và 8ms cho WebSocket message. Tất cả 5 client kết nối đồng thời nhận dữ liệu ổn định mà không gây tắc nghẽn.

5.2.2. Đánh giá hệ thống cảnh báo

Hệ thống cảnh báo phản ứng chính xác và kịp thời: LED đỏ và buzzer kích hoạt trong vòng 2-3 giây kể từ khi AQI vượt ngưỡng 100. Thông báo cảnh báo trên giao diện web xuất hiện ngay trong lần cập nhật WebSocket tiếp theo (tối đa 2 giây sau). Không ghi nhận trường hợp cảnh báo nhầm (false alarm) hoặc bỏ sót cảnh báo trong suốt quá trình thử nghiệm.

5.3. Hạn chế và thách thức

- Cảm biến MQ-135 cần thời gian làm nóng (warm-up) ít nhất 24 giờ để đạt độ ổn định, và chịu ảnh hưởng đáng kể của nhiệt độ và độ ẩm môi trường.
- Hệ thống chưa có khả năng lưu trữ dữ liệu dài hạn do giới hạn bộ nhớ ESP32; dữ liệu bị mất khi khởi động lại.
- Giao diện web chỉ hỗ trợ kết nối trong cùng mạng WiFi, chưa có khả năng truy cập từ xa qua internet.
- Chưa tích hợp chức năng xuất báo cáo dữ liệu dưới dạng CSV hoặc PDF.

6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Kết luận

Tiểu luận đã trình bày toàn bộ quá trình nghiên cứu, thiết kế và hiện thực hóa hệ thống giám sát chất lượng không khí sử dụng ESP32 kết hợp cảm biến MQ-135 và PMS5003. Hệ thống đạt được các mục tiêu đề ra:

- Thành công thu thập và xử lý dữ liệu nồng độ CO và PM2.5 theo thời gian thực với tần suất cập nhật 2 giây.
- Xây dựng giao diện web trực quan, responsive, cung cấp đồng hồ đo, biểu đồ lịch sử và hệ thống cảnh báo theo màu sắc AQI.
- Độ chính xác đạt mức chấp nhận được cho ứng dụng giáo dục và giám sát không chuyên (sai số PM2.5 $\approx 4.2\%$, CO $\approx 12.5\%$).
- Tổng chi phí phần cứng thấp (dưới 400.000 VND), phù hợp triển khai rộng rãi.

Dự án chứng minh rằng với nền tảng ESP32 và các cảm biến tầm trung, hoàn toàn có thể xây dựng các hệ thống IoT giám sát môi trường hữu ích và tiết kiệm chi phí, mở ra nhiều ứng dụng thiết thực trong đời sống và giáo dục.

6.2. Hướng phát triển

Dựa trên kết quả đạt được và những hạn chế được ghi nhận, nhóm đề xuất các hướng phát triển sau cho các phiên bản tiếp theo:

- Tích hợp đám mây và truy cập từ xa: Kết nối ESP32 với nền tảng IoT như ThingSpeak, AWS IoT Core hoặc Firebase để lưu trữ dữ liệu dài hạn và cho phép giám sát từ bất kỳ đâu qua internet.
- Nâng cao độ chính xác cảm biến CO: Bổ sung cảm biến nhiệt độ/độ ẩm DHT22 để hiệu chỉnh theo điều kiện môi trường, hoặc thay thế MQ-135 bằng cảm biến electrochemical chuyên dụng như MQ-7 với độ chọn lọc CO cao hơn.
- Xây dựng mạng cảm biến phân tán: Triển khai nhiều node ESP32 trong toà nhà hoặc khu vực địa lý, đồng bộ dữ liệu về server trung tâm để vẽ bản đồ nhiệt (heatmap) chất lượng không khí.
- Tích hợp học máy: Áp dụng các mô hình Machine Learning nhẹ (TinyML) chạy trực tiếp trên ESP32 để dự đoán xu hướng AQI và cảnh báo sớm trước khi vượt ngưỡng.

- Ứng dụng di động: Phát triển app Android/iOS kết nối Bluetooth hoặc internet để giám sát và nhận thông báo đẩy (push notification) khi chất lượng không khí xấu đi.
- Xuất dữ liệu và báo cáo: Thêm chức năng xuất dữ liệu CSV, vẽ báo cáo PDF tự động và gửi email tóm tắt hàng ngày.

TÀI LIỆU THAM KHẢO

- [1] Espressif Systems. (2023). ESP32 Technical Reference Manual (v5.1). Espressif Systems.
https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf
- [2] Plantower Technology. (2016). PMS5003 Series Data Manual (V2.3). Plantower Co., Ltd.
- [3] Hanwei Electronics Co., Ltd. (2018). MQ-135 Gas Sensor Datasheet. Zhengzhou Winsen Electronics.
- [4] Me, T., Dang, D., & Nguyen, H. (2020). Development of Low-Cost Air Quality Monitoring System Based on IoT. *Journal of Computer Science and Cybernetics*, 36(2), 135-148.
- [5] World Health Organization. (2021). WHO global air quality guidelines: particulate matter (PM_{2.5} and PM₁₀), ozone, nitrogen dioxide, sulfur dioxide and carbon monoxide. WHO Press.
- [6] Khriji, S., El Houssaini, D., Jmal, M. W., Viehweger, C., Kanoun, O., & Cheour, F. (2017). Precision Irrigation based on Wireless Sensor Network. *IET Science, Measurement & Technology*, 11(4), 1-7.
- [7] Me Chin, K., Tan, A. H., & Guan, T. E. (2021). IoT Based Air Quality Monitoring System. *International Journal of Recent Technology and Engineering*, 9(5), 143-148.
- [8] Tổng cục Môi trường - Bộ Tài nguyên và Môi trường Việt Nam. (2022). Hướng dẫn kỹ thuật tính toán và công bố chỉ số chất lượng không khí (AQI). Hà Nội: Nhà xuất bản Tài nguyên Môi trường.
- [9] Me Laconte de Forqueville, G., & Yedder, H. B. (2020). Real-Time Air Quality Monitoring Using IoT. In *Proceedings of the 2020 IEEE International Conference on Big Data* (pp. 4872-4875).
- [10] Arduino ESP32. (2024). ESP32 Arduino Core Documentation. GitHub.
<https://github.com/espressif/arduino-esp32>

- [11] Grigorov, I. (2021). ESPAsyncWebServer: Async HTTP and WebSocket Server for ESP8266 and ESP32. GitHub. <https://github.com/me-no-dev/ESPAsyncWebServer>
- [12] Benoit Blanchon. (2023). ArduinoJson: Efficient JSON library for embedded systems (v6.21). GitHub. <https://github.com/bblanchon/ArduinoJson>